Clase 4 de 5

Google Cloud | The LAB

dbt en la práctica, calidad de datos y orquestación

Ingeniería y Análisis de Datos en Google Cloud

UTN · Facultad Regional Rosario

27 de octubre de 2026

Lo que vimos en la clase 3

Se modela para leer

El warehouse desnormaliza: acepta redundancia a cambio de consultas simples.

Estrella y wide table

Hechos y dimensiones como base, y una wide table encima para consumir.

Cada capa tiene su tarea

Staging limpia cada origen. Marts modela el negocio.

dbt arma el linaje

Cada modelo es un SELECT, y ref() le dice a dbt de qué depende.

Hoy · de la teoría a la práctica

Construir con dbt el proyecto de los pedidos, testear sus datos y ver cómo se orquesta para que corra solo.

El recorrido de hoy

Tres etapas. Las dos primeras con una demo en vivo sobre el proyecto de los pedidos.

1

dbt en la práctica

¿Cómo se construye y se corre un proyecto?

- Demo en vivo: de dbt init a dbt run
- Los comandos y las materializaciones
- El config() y los modelos incrementales

2

Calidad de datos

¿Cómo se sabe si los datos están bien?

- Tests genéricos y singulares
- Demo en vivo: un test que falla
- Qué se testea en cada capa

3

Orquestación

¿Cómo corre todo solo, todos los días?

- Qué hace un orquestador
- Horario o evento, y qué pasa cuando falla
- Las opciones en Google Cloud y fuera de ella

1
dbt en la práctica
Cómo se construye y se corre un proyecto

2
Calidad de datos
Cómo se testean los datos de cada capa

3
Orquestación
Cómo corre el pipeline solo, todos los días

Etapa 1 · dbt en la práctica

Un proyecto de dbt desde cero: crearlo, declarar sus fuentes, escribir los modelos, correrlos y decidir qué objeto crea cada uno en BigQuery.

Los comandos de dbt

Se corren desde la carpeta del proyecto, con el entorno virtual activado.

Comando	Qué hace
dbt run	Construye todos los modelos, en el orden del linaje
dbt test	Corre todos los tests
dbt build	Construye y testea cada modelo; si un test falla, no construye lo que depende de él
dbt compile	Escribe el SQL compilado en target/, sin ejecutarlo
dbt docs generate	Genera la documentación y el grafo de linaje
dbt run -s stg_pedidos+	Construye stg_pedidos y todos los modelos que dependen de él

Las materializaciones

La materialización define qué objeto crea dbt en BigQuery a partir del SELECT de un modelo.

Materialización	Qué crea	Cuándo se calcula	Cuándo conviene
view	Una view	Cada vez que alguien la consulta	Staging: lógica liviana y pocos lectores
table	Una tabla	Cada vez que corre dbt	Marts: muchas lecturas, joins o agregaciones costosas
incremental	Una tabla que crece	Solo sobre las filas nuevas	Tablas grandes que crecen por período

Una view no ocupa almacenamiento, pero cada consulta paga los bytes de las tablas que lee.

Una tabla paga el almacenamiento, y quien la consulta lee solo el resultado.

Views anidadas

Una view sobre otra view no guarda ningún resultado intermedio. Cada lectura recalcula toda la cadena.

Views anidadas

`pedidos`
tabla raw

↓

`v_stg`
view: guarda una query

↓

`v_ventas`
view: guarda una query

Cada consulta a v_ventas ejecuta las dos queries y paga los bytes de raw.

Se elige cuando

- Se lee poco
- Tiene que mostrar siempre lo último de raw

Tablas materializadas

`pedidos`
tabla raw

↓

`stg_pedidos`
view

↓

`ventas_diarias`
table: guarda el resultado

Cada consulta a ventas_diarias lee y paga solo el resultado.

Se elige cuando

- Se lee mucho, o tiene joins y agregaciones costosas
- Necesita partitioning y clustering propios

La configuración de un modelo

La materialización y el diseño físico se declaran en código: para una carpeta en dbt_project.yml, o para un modelo en su config().

Por carpeta · dbt_project.yml

```yaml
models:
  pedidos:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

Por modelo · config() al principio del .sql

```
{{ config(
    materialized='table',
    partition_by={
        'field': 'fecha_carga',
        'data_type': 'date',
        'granularity': 'day'
    },
    cluster_by=['medio_pago']
) }}
```

Si un modelo tiene config(), esa configuración reemplaza a la de su carpeta.

Son las decisiones de la clase 2. Quedan escritas junto al modelo y se versionan con él.

Los modelos incrementales

Un modelo incremental procesa solo las filas nuevas y las agrega a la tabla existente, en lugar de reconstruirla entera en cada corrida.

`table`

Cada corrida lee todo el historial y reescribe la tabla.

`incremental`

La primera corrida construye la tabla. Las siguientes leen solo lo posterior a la última fila cargada y lo agregan con MERGE.

```
{{ config(
    materialized='incremental',
    unique_key='pedido_id'
) }}
select ...
from {{ ref('stg_pedidos') }}
{% if is_incremental() %}
where fecha > (
    select max(fecha) from {{ this }}
)
{% endif %}
```

dbt implementa los modelos incrementales con MERGE, y el sandbox no permite DML. En el curso se ven como concepto y quedan fuera del TP.

Demo en vivo · de dbt init a dbt run

El proyecto de los pedidos, desde dbt init. El proyecto completo queda en el repositorio del curso, en clase-4/demo-dbt.

1 **Crear el proyecto**
`dbt init pedidos`

2 **Probar la conexión**
`dbt debug`

3 **Copiar los modelos**
`models/ y tests/`

4 **Compilar**
`dbt compile -s stg_pedidos`

5 **Construir todo**
`dbt run`

6 **Correr una parte**
`dbt run -s fct_pedidos+`

7 **Ver el linaje**
`dbt docs generate · dbt docs serve`

8 **Cambiar la materialización**
`config(materialized='view')`

9 **Declarar el diseño físico**
`config(cluster_by=…, partition_by=…)`

10 **Un modelo que falla**
`dbt run · dbt retry`

10:00

Receso

1
dbt en la práctica
Cómo se construye y se corre un proyecto

2
Calidad de datos
Cómo se testean los datos de cada capa

3
Orquestación
Cómo corre el pipeline solo, todos los días

Etapa 2 · Calidad de datos

Qué es un test en dbt, cómo se lee uno que falla y qué conviene testear en cada capa.

Los tests de datos

Un test es una query que devuelve las filas que no deberían existir. Pasa cuando no devuelve ninguna.

Genéricos

Vienen con dbt y se declaran en el .yml de cada modelo: unique, not_null, accepted_values y relationships.

```yaml
columns:
  - name: pedido_id
    data_tests:
      - unique
      - not_null
  - name: medio_pago
    data_tests:
      - accepted_values:
          arguments:
            values: ['tarjeta', 'efectivo',
                'transferencia']
```

Singulares

Un archivo .sql en tests/ con la query que busca las filas incorrectas. Sirven para las reglas propias de cada negocio.

tests/assert_codigo_postal_cuatro_digitos.sql

```sql
select pedido_id, codigo_postal
from {{ ref('stg_pedidos') }}
where not regexp_contains(
    codigo_postal, r'^\d{4}$'
)
```

Qué se testea en cada capa

Cada capa tiene su propia promesa, y los tests verifican que se cumpla.

Capa	Qué se testea	Ejemplo
Raw	Que los datos lleguen y estén al día	La última carga es de las últimas 24 horas
Staging	Claves, valores obligatorios y formatos	pedido_id único y no nulo; medio_pago dentro de los valores aceptados
Marts	Granularidad y reglas de negocio	Una sola fila por fecha y medio de pago; el monto neto nunca supera al bruto

Un test puede cortar la ejecución o solo avisar: severity: error o warn.

El test de granularidad de un mart es el que detecta un join que multiplicó filas.

Demo en vivo · un test que falla

Los tests del proyecto de los pedidos. Uno falla, y la corrección va en staging.

1. **Correr los tests**
   `dbt test · 14 PASS y 1 FAIL`
2. **Bajar la severidad**
   `config(severity='warn')`
3. **Guardar las filas que fallan**
   `dbt test -s stg_pedidos --store-failures`
4. **Construir y testear**
   `dbt build · 1 ERROR y 15 SKIP`
5. **Corregir en staging**
   `regexp_extract(codigo_postal, r'\d{4}')`
6. **Todo en verde**
   `dbt build · 21 PASS`

1
dbt en la práctica
Cómo se construye y se corre un proyecto

2
Calidad de datos
Cómo se testean los datos de cada capa

3
Orquestación
Cómo corre el pipeline solo, todos los días

Etapa 3 · Orquestación

Cómo se programa el pipeline para que corra solo, en orden, y quién se entera cuando falla. Hoy la teoría; la demo, en la clase 5.

El pipeline corrido a mano

Hoy el proyecto de los pedidos corre cuando alguien escribe dbt build. En producción tiene que correr todos los días, con o sin alguien mirando.

05:30
Llega pedidos.csv del sistema de ventas

→

06:00
Se carga en la capa raw

→

06:15
Corre dbt build

→

08:00
El dashboard muestra las ventas de ayer

¿Quién lo corre el domingo?

Si depende de una persona, el pipeline corre los días que esa persona trabaja.

¿Qué pasa si el archivo llega a las 7?

La carga de las 6:00 encuentra el archivo de ayer, y el dashboard muestra datos viejos.

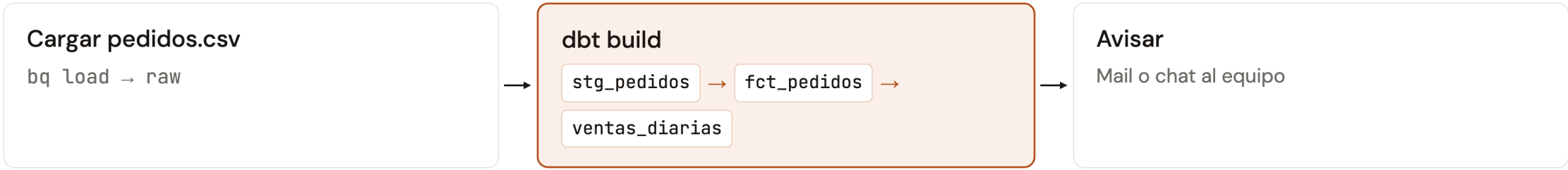
¿Quién se entera si falla?

Si dbt build falla a las 6:15, el primero en notarlo es quien abre el dashboard.

Un orquestador es la pieza que responde estas tres preguntas.

El orquestador

Un orquestador decide cuándo corre cada paso del pipeline, en qué orden, y qué hacer cuando uno falla.



dbt ordena los modelos dentro de un paso. El orquestador ordena los pasos.

Programa

Corre a una hora fija o cuando llega un dato nuevo.

Ordena

Un paso arranca solo cuando el anterior terminó bien.

Reintenta

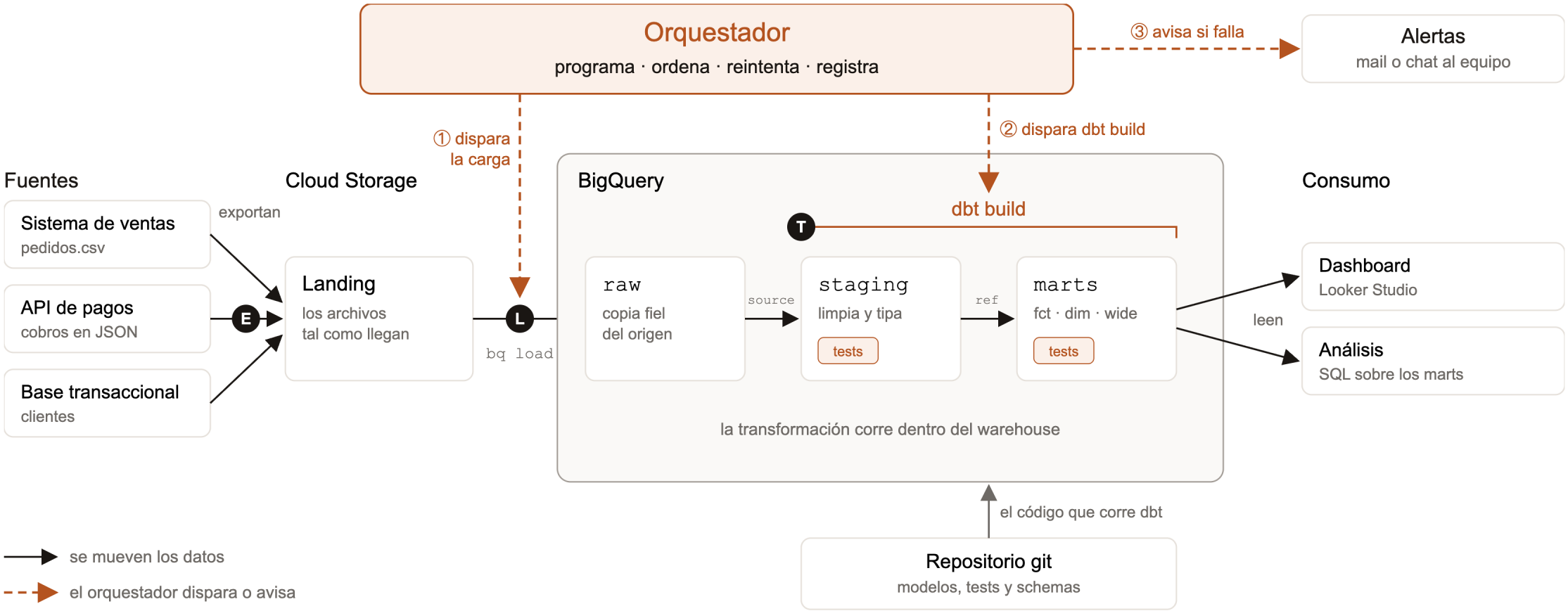
Vuelve a correr un paso que falló por un error pasajero.

Registra y avisa

Guarda cada corrida con su log y alerta cuando algo falla.

La arquitectura ELT con orquestador

Los datos se mueven de izquierda a derecha. El orquestador, arriba, decide cuándo arranca cada paso.



Horario o evento

Un pipeline arranca de una de dos maneras: a una hora fija o cuando pasa algo, como la llegada de un archivo.

Por horario

```
0 6 * * *
| | | | └ día de la semana
| | | └── mes
| | └──── día del mes
| └────── hora
└──────── minuto
```

Una expresión cron: todos los días a las 6:00.

A favor: es simple y se sabe cuándo va a correr.

En contra: si el archivo llega tarde, corre igual sobre los datos de ayer.

Por evento

```
pedidos.csv llega al bucket
→ arranca la carga
→ arranca dbt build
```

Cada paso arranca cuando termina el anterior, sin hora fija.

A favor: corre apenas hay datos nuevos.

En contra: tiene más piezas, y si el archivo nunca llega, no corre nada ni avisa.

Con horario, conviene verificar la frescura de raw antes de dbt build: dbt source freshness falla si la última carga es vieja.

Las corridas que fallan

Un pipeline que corre todos los días va a fallar alguna vez. Lo que importa es qué pasa después.

Reintentos

Un corte de red o un límite de cuota se resuelven volviendo a correr. Un error en los datos no: el reintento falla igual.

Alertas

El aviso llega a quien puede arreglarlo, con el paso que falló y su log.

Idempotencia

Correr dos veces el mismo día deja el mismo resultado. Un modelo table lo cumple porque reemplaza la tabla entera.

Retomar

No hace falta correr todo de nuevo: dbt retry arranca desde el modelo que falló, como en la demo.

Si correr dos veces da el mismo resultado, reintentar es seguro.

Las opciones para orquestar

De la más simple a la más completa. Las tres primeras son servicios de Google Cloud.

Herramienta	Qué es	Conviene cuando	Costo
Cloud Scheduler + Cloud Run	Un cron administrado que arranca un contenedor con dbt build	Un solo pipeline con horario fijo	3 jobs gratis por cuenta; después USD 0,10 por job al mes
Workflows	Encadena pasos que llaman servicios, con reintentos y condiciones	Varios pasos dentro de Google Cloud	5.000 pasos internos gratis por mes
Managed Service for Apache Airflow	Airflow administrado, antes Cloud Composer. Los pipelines se escriben en Python	Muchos pipelines con dependencias entre equipos	Entorno siempre encendido: cientos de USD por mes
Dagster	Orquestador open source en Python. Cada modelo de dbt aparece como un asset propio	Pipelines en Python que combinan dbt con otros pasos	Open source gratis; Dagster+ desde USD 10 por mes
dbt platform	Jobs programados que corren dbt build	El pipeline es solo dbt	Plan gratuito para un desarrollador
GitHub Actions	Un workflow con cron en el mismo repositorio	Equipo chico con el código en GitHub	Gratis en repositorios públicos

En la clase 5, la demo: dbt build programado para que corra solo.

Gracias

Preguntas

Google Cloud | The LAD